Relay on Codex: running coding-agent plans across ChatGPT usage windows

cube-digital · relay codex-math v0.1 · October 2026 · Codex CLI 0.163.0, read with REA 6.7.0 · github.com/cube-digital/relay

Relay turns a written plan into parallel work for the cheapest model that can do each task and keeps the plan running across usage windows. This page is the Codex counterpart of the relay whitepaper: what the Codex harness meters and exposes (recovered from the binary, not assumed), what that changes in relay's mechanism, and the model behind the Codex forecast and its calibration. The harness facts and their evidence are in codex-math/config/codex-harness.json; the model code in codex-math/lib/.

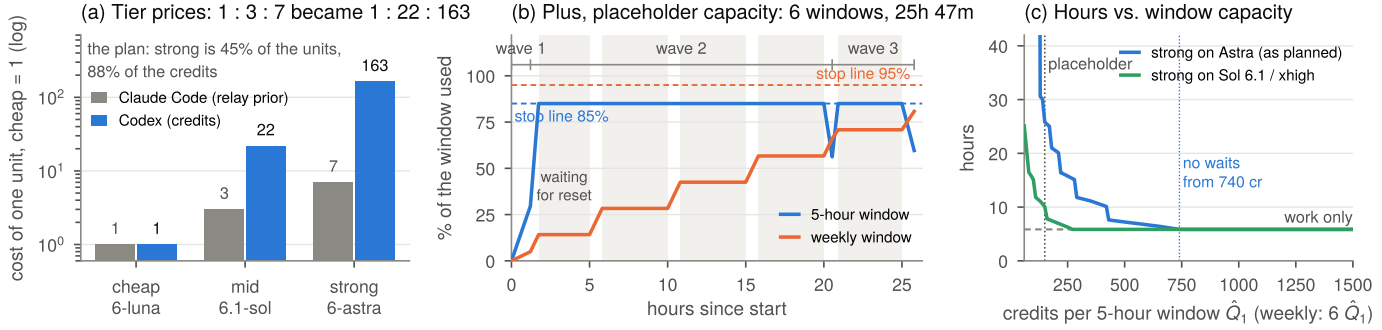


Figure 1. The relay whitepaper's 30-task plan (7 lanes, 3 waves, 108.4 units, 4.7 expected fix rounds) on Codex at the priors. (a) Credits per unit by tier: Luna : Sol 6.1 : Astra is 1 : 21.8 : 163 where relay's Claude prior was 1 : 3 : 7; the plan's strong tasks are 45% of its units and 88% of its 727 credits. (b) The walk on a Plus account (5-hour and weekly windows) at the placeholder capacities $Q_1 = 150$, $Q_2 = 900$ credits: six 5-hour windows, 5 h 51 min of work, 25 h 47 min in all. (c) Hours against the capacity $\hat{Q}_1$ that calibration supplies first: no pause above 740 credits; with the strong tier on Sol 6.1/xhigh the plan costs 258 credits and takes 10 h 05 min at the placeholder. Computed by data.mjs with the model code.

1 What the harness meters

Read from @openai/codex 0.163.0 with REA: process captures of the binary (help, doctor, exec --json, the TUI, the app-server over stdio), its own protocol schema (app-server generate-json-schema) and model catalog (debug models), and the strings of its 15 MB .rodata; the credit rates are OpenAI's published ones.

- **Windows are plan-shaped, not fixed.** A reading is a RateLimitSnapshot: primary and secondary windows with usedPercent, resetsAt and windowDurationMins, plus planType, credits, a per-pool view keyed by limit_id, and rateLimitReachedType. Plus and Business meter a 5-hour and a weekly window; Pro only the weekly one. Readings come from account/rateLimits/read (and /updated) on the app-server and from the token_count event in each session's rollout file; API-key sessions get none.
- **Credits are the unit.** The backend estimates every thread in credits (/wham/usage/thread-estimates/query, grouped by model, effort and speed with token counts). Published rates per million tokens, input / cached / output: Luna 2.5 / 0.25 / 12.5, Sol 6.1 50 / 2.5 / 250, Astra 250 / 25 / 1250. Fast meters 2.5× the included usage (2× purchased credits) for a 1.5–2× speedup; Ultrafast 8×.
- **Tokens are attributable.** Every subagent is its own thread: state_5.sqlite holds its model, effort and token total, thread_spawn_edges its parent; exec --json ends with turn.completed.usage. The effective context is 95% of 272 k = 258 400 tokens; tool output is cut at 10 000 tokens; compaction is configurable and hookable (PreCompact, PostCompact).
- **Levers relay did not have.** Spawns take model, effort and speed; [agents] sets the subagent defaults; a PreToolUse hook can rewrite a spawn_agent call through updatedInput (the Codex form of relay's mod); reset credits restore every window at once (account/rateLimitResetCredit/consume); a thread *goal* with a token budget keeps an idle thread going until it is budget_limited or usage_limited (relay's marathon, built in).

2 What changes in the mechanism

Tiers keep their roles, but the ladder is two orders of magnitude, so the cheap tier is nearly free: Luna costs 1/22 of Sol per unit and still 1/7 at three times the turns, and Astra is spent only where the plan says strong. Budget gates degrade in price order: speed to standard, effort to medium, then strong to mid; the architect is exempt at tight. A reset credit is spent when the next pause exceeds an hour. The ledger stamp carries the windows the account has and the credits spent: [primary 23.4% resets 16:30 · secondary 41% · credits 12.5].

3 The model

Units. As in relay, task t weighs $w_t = T(\text{tier}_t)R(\text{risk}_t)$ with $T = (1,3,7)$, $R = (0.8,1,1.4)$, read now as relative token volume. On tier τ a unit processes ν_τ tokens (prior 70 k) in shares $\alpha = (0.12, 0.85, 0.03)$ of uncached input, cached input and output; with the model's rates p_τ, the speed multiplier s_τ and the effort multiplier e_τ on output, a unit costs

$$c_\tau = s_\tau \nu_\tau \left(\alpha_{\text{in}} p_\tau^{\text{in}} + \alpha_{\text{c}} p_\tau^{\text{c}} + \alpha_{\text{out}} e_\tau p_\tau^{\text{out}}\right)/10^6$$

credits: 0.062, 1.36 and 10.15 at the priors. Lane l costs $C_l = \sum_{t\in l} w_t c_{\tau(t)}$.

Fix rounds. Unchanged: lane l needs a fix round with probability $p_l \in \{0.2, 0.5, 0.8\}$ re-working $\kappa = 0.35$ of it, so $\tilde{C}_l = C_l(1+\kappa p_l)$, $\mathrm{E}[C] = \sum_l \tilde{C}_l = 727$ and $\mathrm{Var}[C] = \kappa^2 \sum_l C_l^2 p_l(1-p_l)$, SD = 49; the exact distribution over the 2^7 outcomes agrees.

Windows. Window j holds Q_j credits, so the plan uses $U_j = \mathrm{E}[C]/Q_j$ of it. The walk runs a wave while every window is below its stop line (85% primary, 95% secondary), spends credits and time linearly, pauses at a line until that window resets, rolls windows over on their own schedule, and spends a reset credit instead of a pause when one is available and the pause is long. Wave time is $m\,M_k$ with M_k the makespan of the lanes' expected units on four slots; Fast divides a lane's minutes by its speedup and multiplies its credits by 2.5, which prices the trade per wave (Fig. 1 plan: wave 1 saves 32 min for 33 credits, wave 2 saves 93 min for 603).

Calibration has two layers that never share a parameter. Capacity: between two readings with no reset, the credits spent and the points the window moved give $\hat{Q}_j = 100 \sum \Delta C / \sum \Delta u_j$, a ratio estimator over all such intervals, independent of the plan. Price per unit: every lane thread's credits are known, so $\hat{c}_\tau = \sum C / \sum w$ over the threads on tier τ; with only wave totals, non-negative least squares of ΔC_k on the units per tier. Time: $\hat{m} = \sum \Delta t_k / \sum M_k$ as before. Each estimate replaces its prior from the first sample; $\hat{Q}_j$ is the number that moves the hours (Fig. 1c), and one clean wave gives it.

4 What is not claimed

- Q_j, ν_τ, α, e_τ, T, R, p_l and κ are priors; the capacities were scaled from OpenAI's "messages per five hours" and must be calibrated before a Codex forecast means anything. used_percent is the backend's and an integer, so capacity samples carry credits across readings that did not move it; the rates may change with the catalog.
- Not recovered from the binary: the default auto-compaction limit (null in the catalog, computed in code) and the default subagent concurrency; both are configurable. Codex caches input only while the prefix is stable, so a mid-thread model, effort or speed change re-reads the context at the uncached rate.
- Lanes are independent and front-load usage less than real runs do; the walk ignores spend controls and pools other than the primary limit_id.
- Savings are not measured yet. Next: ten calibrated waves, then an A/B of the same plans on Codex plain, with the Sol 6.1 strong tier, and with relay.

Evidence: codex-math/evidence/ (REA captures, protocol schemas, catalog, hook schemas, SQLite DDL, .rodata strings with offsets). Docs: learn.chatgpt.com/docs/pricing, /docs/agent-configuration/subagents, /docs/config-file/config-reference; help.openai.com/20001507. Rebuild: codex-math/docs/whitepaper/README.md.